

Machine Learning Assessment 2

Varun Wadhwa

SN - 2024001633

Abstract

This project focuses on developing an object detection system using the YOLOv8 (You Only Look Once) deep learning model to identify and classify different types of bees and related objects, including Queen Bee, Worker Bee, Drone Bee, and Varroa Mite. The dataset was prepared and managed using Roboflow, which provided preprocessing and augmentation capabilities to improve model performance. Multiple training experiments were conducted by adjusting hyperparameters such as learning rate, batch size, and number of epochs. The trained model achieved strong performance with high precision and mean Average Precision (mAP), demonstrating effective object detection capabilities. Additionally, an object counting system was implemented to automatically count detected objects in test images, showcasing the model's practical applications in real-world scenarios such as hive monitoring and pest detection.

Introduction

Object detection is a key area of computer vision that enables machines to identify and locate objects within images. It has significant applications in agriculture, environmental monitoring, and biological research. In this project, the focus is on detecting and classifying different types of bees and harmful pests such as Varroa mites, which are known to impact bee health. The YOLOv8 model was selected due to its efficiency and accuracy in real-time object detection tasks. A dataset containing various bee images was used, including different hive conditions and object variations. The dataset was processed using Roboflow to ensure consistency and improve model generalisation. The project involved training multiple models with different hyperparameters to analyse performance improvements. Evaluation metrics such as precision, recall, and mean Average Precision were used to measure model effectiveness. Furthermore, an object counting system was implemented to extend the functionality of the model, allowing automated counting of detected objects in images.

Data Augmentation

Data augmentation was applied to improve the diversity of the dataset and enhance the model's ability to generalise to different real-world conditions. Augmentation techniques such as horizontal flipping, colour adjustments, and slight transformations were used.

These techniques help simulate variations in lighting, orientation, and environmental conditions, allowing the model to learn more robust feature representations. For example, images were flipped horizontally to simulate different viewing angles, and colour adjustments were applied to account for varying lighting conditions in hive environments.

Data augmentation is particularly important in this project due to the similarity between bee types and the presence of small objects such as Varroa mites. By increasing variation in the training data, the model is better able to distinguish between different classes and improve detection performance.

Task 1 - Dataset & Preprocessing

Installing Libraries

```
# Install required libraries
!pip install ultralytics roboflow --quiet
```

```
# Import libraries
import os
import torch
from ultralytics import YOLO
```

```

 41.2/41.2 kB 4.1 MB/s eta 0:00:00
 1.3/1.3 MB 64.5 MB/s eta 0:00:00
249.2/249.2 kB 26.9 MB/s eta 0:00:00
 66.8/66.8 kB 7.3 MB/s eta 0:00:00
 49.9/49.9 MB 15.2 MB/s eta 0:00:00
 1.5/1.5 MB 68.7 MB/s eta 0:00:00
 5.5/5.5 MB 106.2 MB/s eta 0:00:00
```

Creating new Ultralytics Settings v0.0.6 file 

View Ultralytics Settings with 'yolo settings' or at '/root/.config/Ultralytics/settings.json'

Update Settings with 'yolo settings key=value', i.e. 'yolo settings runs_dir=path/to/dir'. For help see <https://docs.ultralytics.com/quickstart/#ultralytics-settings>.

This project uses the Ultralytics YOLOv8 framework for object detection. PyTorch is used as the backend deep learning framework. These tools provide efficient and real-time object detection capabilities.

Loading Dataset

```
from roboflow import Roboflow
```

```
rf = Roboflow(api_key="OLvbopQ8iQHStAzvkhJL")
project = rf.workspace("varuns-workspace-u056q").project("bee-object-detection-final")
dataset = project.version(3).download("yolov8")
```

```
loading Roboflow workspace...
```

```
loading Roboflow project...
```

```
Downloading Dataset Version Zip in Bee-Object-Detection-Final-3 to yolov8:: 100%|██████████| 277651/277651 [00:17<00:00, 15583.00it/s]
```

```
Extracting Dataset Version Zip to Bee-Object-Detection-Final-3 in yolov8:: 100%|██████████| 13175/13175 [00:01<00:00, 7172.16it/s]
```

The dataset was imported using Roboflow, which provides tools for dataset management, preprocessing, and exporting in YOLO format.

```
model = YOLO("yolov8n.pt")
```

```
Downloading https://github.com/ultralytics/assets/releases/download/v8.4.0/yolov8n.pt to 'yolov8n.pt': 100% ————— 6.2MB 294.7MB/s 0.0s
```

The YOLOv8n model was selected due to its lightweight architecture and efficient performance, making it suitable for training in a limited environment such as Google Colab.

RUN 1

```
results = model.train(  
    data=f"{dataset.location}/data.yaml",  
    epochs=80,  
    batch=32,  
    imgsz=640,  
    lr0=0.0005,  
    optimizer="Adam",  
    momentum=0.937,  
    weight_decay=0.0005,  
    patience=20  
)
```

59/80	7.13G	1.033	0.6795	1.141	31	640: 100% 191/191 1.9it/s 1:41
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 5/5 1.5it/s 3.4s
	all	299	1559	0.799	0.743	0.783 0.509
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
60/80	7.13G	1.03	0.6727	1.134	31	640: 100% 191/191 1.9it/s 1:40
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 5/5 1.3it/s 4.0s
	all	299	1559	0.805	0.755	0.799 0.515
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
61/80	7.13G	1.016	0.6634	1.134	31	640: 100% 191/191 1.9it/s 1:39
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 5/5 1.0it/s 4.8s
	all	299	1559	0.811	0.747	0.805 0.52
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
62/80	7.13G	1.02	0.6611	1.132	31	640: 100% 191/191 1.9it/s 1:39
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 5/5 1.6it/s 3.2s
	all	299	1559	0.819	0.748	0.811 0.525
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
63/80	7.13G	1.018	0.6577	1.132	31	640: 100% 191/191 1.9it/s 1:40
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 5/5 1.3it/s 3.8s
	all	299	1559	0.81	0.745	0.794 0.515
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
64/80	7.13G	1.006	0.6469	1.125	31	640: 100% 191/191 1.9it/s 1:39
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 5/5 1.1it/s 4.6s
	all	299	1559	0.813	0.726	0.79 0.517
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
65/80	7.13G	1.01	0.6449	1.125	31	640: 100% 191/191 1.9it/s 1:41
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 5/5 1.3it/s 4.0s
	all	299	1559	0.805	0.749	0.794 0.514

Hyperparameters

RUN 2

```
model = YOLO("yolov8n.pt")

model.train(
    data=f"{dataset.location}/data.yaml",
    epochs=30,
    batch=16,
    lr0=0.001
)
```

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
24/30	3.93G	1.127	0.774	1.217	15	640: 100% 382/382 4.0it/s 1:36
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 10/10 2.5it/s 4.0s
	all	299	1559	0.769	0.733	0.766 0.496
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
25/30	3.93G	1.112	0.7616	1.209	15	640: 100% 382/382 4.0it/s 1:36
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 10/10 3.8it/s 2.6s
	all	299	1559	0.786	0.735	0.781 0.501
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
26/30	3.93G	1.094	0.7381	1.198	15	640: 100% 382/382 4.0it/s 1:37
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 10/10 2.9it/s 3.5s
	all	299	1559	0.786	0.741	0.776 0.498
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
27/30	3.93G	1.096	0.729	1.195	15	640: 100% 382/382 4.0it/s 1:35
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 10/10 3.7it/s 2.7s
	all	299	1559	0.817	0.725	0.793 0.506
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
28/30	3.93G	1.077	0.7126	1.186	15	640: 100% 382/382 3.9it/s 1:38
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 10/10 3.3it/s 3.0s
	all	299	1559	0.803	0.74	0.787 0.501
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
29/30	3.93G	1.057	0.6915	1.177	15	640: 100% 382/382 4.0it/s 1:35
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 10/10 3.4it/s 2.9s
	all	299	1559	0.823	0.735	0.794 0.516
Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size
30/30	3.93G	1.05	0.6855	1.168	15	640: 100% 382/382 3.9it/s 1:37
	Class	Images	Instances	Box(P	R	mAP50 mAP50-95): 100% 10/10 3.6it/s 2.8s
	all	299	1559	0.815	0.734	0.795 0.511

30 epochs completed in 0.867 hours.

Optimizer stripped from /content/runs/detect/train/weights/last.pt, 6.2MB

Optimizer stripped from /content/runs/detect/train/weights/best.pt, 6.2MB

Validating /content/runs/detect/train/weights/best.pt...

Ultralytics 8.4.60 🚀 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

Model summary (fused): 73 layers, 3,006,428 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95): 100%	10/10 2.3it/s 4.3s
all	299	1559	0.822	0.736	0.796	0.516	
Drone-Bee	40	188	0.807	0.761	0.836	0.623	
Queen-Bee	107	145	0.897	0.876	0.908	0.644	
Varroa-Mite	135	415	0.75	0.472	0.569	0.245	
Worker-Bee	56	811	0.833	0.836	0.872	0.553	

Speed: 0.2ms preprocess, 2.6ms inference, 0.0ms loss, 2.7ms postprocess per image

Results saved to /content/runs/detect/train

ultralytics.utils.metrics.DetMetrics object with attributes:

ap class index: array([0, 1, 2, 3])

```
model = YOLO("yolov8n.pt")
```

```
model.train(  
    data=f"{dataset.location}/data.yaml",  
    epochs=50,  
    batch=16,  
    lr0=0.0005  
)
```

```

keys: ['metrics/precision(B)', 'metrics/recall(B)', 'metrics/mAP50(B)', 'metrics/mAP50-95(B)']
maps: array([ 0.64211, 0.6617, 0.25295, 0.5633])
names: {0: 'Drone-Bee', 1: 'Queen-Bee', 2: 'Varroa-Mite', 3: 'Worker-Bee'}
nt_per_class: array([188, 145, 415, 811])
nt_per_image: array([ 40, 107, 135, 56])
results_dict: {'metrics/precision(B)': 0.821698141048352, 'metrics/recall(B)': 0.7408195593064927, 'metrics/mAP50(B)': 0.8199018306812356, 'metrics/mAP50-95(B)': 0.530016344980492, 'fitness': 0.530016344980492}
save_dir: PosixPath('/content/runs/detect/train')
speed: {'preprocess': 0.274262401339409, 'inference': 2.673870267557942, 'loss': 0.0003961973214908628, 'postprocess': 4.956434722406504}
stats: {'tn': [1, 'conf': [1, 'pred cls': [1, 'target cls': [1, 'target img': [1]}

```

Optimizers

```

from ultralytics import YOLO

optimizers = ["SGD", "Adam", "AdamW"]
results_opt = []

for opt in optimizers:
    print(f"\nTraining with optimizer: {opt}\n")

    model = YOLO("yolov8n.pt")

    model.train(
        data=f"{dataset.location}/data.yaml",
        epochs=15,
        batch=16,
        lr0=0.001,
        optimizer=opt,
        verbose=False
    )

    metrics = model.val()

    results_opt.append({
        "Optimizer": opt,
        "mAP50": metrics.box.map50,
        "Precision": metrics.box.mp,
        "Recall": metrics.box.mr
    })

print(results_opt)

```

	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	10/10	3.1it/s	3.2s
	all	299	1559	0.681	0.681	0.699	0.435				

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size					
12/15	3.54G	1.207	0.9317	1.305	15	640:	100%	382/382	4.0it/s	1:36	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	10/10	2.7it/s	3.7s
	all	299	1559	0.778	0.699	0.76	0.473				

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size					
13/15	3.54G	1.191	0.9001	1.289	15	640:	100%	382/382	3.9it/s	1:38	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	10/10	2.6it/s	3.9s
	all	299	1559	0.795	0.685	0.757	0.48				

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size					
14/15	3.54G	1.167	0.8594	1.261	15	640:	100%	382/382	3.9it/s	1:38	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	10/10	3.3it/s	3.0s
	all	299	1559	0.741	0.714	0.756	0.486				

Epoch	GPU_mem	box_loss	cls_loss	df1_loss	Instances	Size					
15/15	3.54G	1.145	0.83	1.257	15	640:	100%	382/382	3.9it/s	1:39	
	Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	10/10	3.7it/s	2.7s
	all	299	1559	0.764	0.708	0.761	0.49				

15 epochs completed in 0.433 hours.

Optimizer stripped from /content/runs/detect/train-6/weights/last.pt, 6.2MB

Optimizer stripped from /content/runs/detect/train-6/weights/best.pt, 6.2MB

Validating /content/runs/detect/train-6/weights/best.pt...

Ultralytics 8.4.60 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

Model summary (fused): 73 layers, 3,006,428 parameters, 0 gradients, 8.1 GFLOPs

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	10/10	1.9it/s	5.4s
all	299	1559	0.764	0.706	0.76	0.489				

Speed: 0.2ms preprocess, 3.0ms inference, 0.0ms loss, 5.5ms postprocess per image

Results saved to /content/runs/detect/train-6

Ultralytics 8.4.60 Python-3.12.13 torch-2.11.0+cu128 CUDA:0 (Tesla T4, 14913MiB)

Model summary (fused): 73 layers, 3,006,428 parameters, 0 gradients, 8.1 GFLOPs

val: Fast image access (ping: 0.0±0.0 ms, read: 1945.3±404.0 MB/s, size: 72.4 KB)

val: Scanning /content/Bee-Object-Detection-Final-3/valid/labels.cache... 299 images, 6 backgrounds, 0 corrupt: 100% 299/299 104.5Mit/s 0.0s

WARNING ⚠ Box and segment counts should be equal, but got len(segments) = 163, len(boxes) = 1559. To resolve this only boxes will be used and all segments will be re

Class	Images	Instances	Box(P	R	mAP50	mAP50-95):	100%	19/19	3.2it/s	6.0s
all	299	1559	0.764	0.707	0.761	0.49				
Drone-Bee	40	188	0.774	0.766	0.837	0.625				
Queen-Bee	107	145	0.881	0.818	0.884	0.584				
Varroa-Mite	135	415	0.587	0.431	0.463	0.216				
Worker-Bee	56	811	0.814	0.814	0.86	0.534				

Speed: 3.6ms preprocess, 4.2ms inference, 0.0ms loss, 2.5ms postprocess per image

Results saved to /content/runs/detect/val-4

[{'Optimizer': 'SGD', 'mAP50': np.float64(0.7059413742290119), 'Precision': np.float64(0.677647425431638), 'Recall': np.float64(0.6829817947751768)}, {'Optimizer': 'Ad

Results of using different Optimizers

Using epoch 15 due to less time and batch size = 16

Optimizer	mAP50	Precision	Recall
SGD	0.7059413742290119	0.677647425431638	0.6829817947751768
Adam	0.7523238371903032	0.7599293994790653	0.7128513817352319
AdamW	0.7609916474622832	0.764183099226548	0.7073600040183438

Optimizer Comparison and Analysis

The experiment comparing three optimizers (SGD, Adam, and AdamW) shows clear differences in model performance. SGD achieved the lowest performance, with reduced mAP, precision, and recall, indicating slower convergence and less effective optimisation.

Adam significantly improved the results by using adaptive learning rates, which allowed faster and more stable training. This resulted in higher mAP and better detection performance compared to SGD. AdamW achieved the best overall performance, with the highest mAP@50 of 0.761 and the highest precision. This is because AdamW improves regularisation by effectively handling weight decay, reducing overfitting and improving generalisation.

Although Adam achieved slightly higher recall than AdamW, AdamW provided the best balance between accuracy and stability. Therefore, AdamW was selected as the most suitable optimizer for this task.

⌵

Metric Evaluation

```
metrics = model.val()  
print(metrics)
```

0.55255,	0.55355,	0.55455,	0.55556,	0.55656,	0.55756,	0.55856,	0.55956,	0.56056,	0.56156,	0.56256,	0.56356,	
0.57658,	0.57758,	0.57858,	0.57958,	0.58058,	0.58158,	0.58258,	0.58358,	0.58458,	0.58559,	0.58659,	0.58759,	
0.6006,	0.6016,	0.6026,	0.6036,	0.6046,	0.60561,	0.60661,	0.60761,	0.60861,	0.60961,	0.61061,	0.61161,	
0.62462,	0.62563,	0.62663,	0.62763,	0.62863,	0.62963,	0.63063,	0.63163,	0.63263,	0.63363,	0.63463,	0.63564,	
0.64865,	0.64965,	0.65065,	0.65165,	0.65265,	0.65365,	0.65465,	0.65566,	0.65666,	0.65766,	0.65866,	0.65966,	
0.67267,	0.67367,	0.67467,	0.67568,	0.67668,	0.67768,	0.67868,	0.67968,	0.68068,	0.68168,	0.68268,	0.68368,	
0.6967,	0.6977,	0.6987,	0.6997,	0.7007,	0.7017,	0.7027,	0.7037,	0.7047,	0.70571,	0.70671,	0.70771,	
0.72072,	0.72172,	0.72272,	0.72372,	0.72472,	0.72573,	0.72673,	0.72773,	0.72873,	0.72973,	0.73073,	0.73173,	
0.74474,	0.74575,	0.74675,	0.74775,	0.74875,	0.74975,	0.75075,	0.75175,	0.75275,	0.75375,	0.75475,	0.75576,	
0.76877,	0.76977,	0.77077,	0.77177,	0.77277,	0.77377,	0.77477,	0.77578,	0.77678,	0.77778,	0.77878,	0.77978,	
0.79279,	0.79379,	0.79479,	0.7958,	0.7968,	0.7978,	0.7988,	0.7998,	0.8008,	0.8018,	0.8028,	0.8038,	
0.81682,	0.81782,	0.81882,	0.81982,	0.82082,	0.82182,	0.82282,	0.82382,	0.82482,	0.82583,	0.82683,	0.82783,	
0.84084,	0.84184,	0.84284,	0.84384,	0.84484,	0.84585,	0.84685,	0.84785,	0.84885,	0.84985,	0.85085,	0.85185,	
0.86486,	0.86587,	0.86687,	0.86787,	0.86887,	0.86987,	0.87087,	0.87187,	0.87287,	0.87387,	0.87487,	0.87588,	
0.88889,	0.88989,	0.89089,	0.89189,	0.89289,	0.89389,	0.89489,	0.8959,	0.8969,	0.8979,	0.8989,	0.8999,	
0.91291,	0.91391,	0.91491,	0.91592,	0.91692,	0.91792,	0.91892,	0.91992,	0.92092,	0.92192,	0.92292,	0.92392,	
0.93694,	0.93794,	0.93894,	0.93994,	0.94094,	0.94194,	0.94294,	0.94394,	0.94494,	0.94595,	0.94695,	0.94795,	
0.96096,	0.96196,	0.96296,	0.96396,	0.96496,	0.96597,	0.96697,	0.96797,	0.96897,	0.96997,	0.97097,	0.97197,	
0.98498,	0.98599,	0.98699,	0.98799,	0.98899,	0.98999,	0.99099,	0.99199,	0.99299,	0.99399,	0.99499,	0.996,	
[	0.047929,	0.047946,	0.072075, ...,	1,	1,	1],						
[	0.081409,	0.08148,	0.10488, ...,	1,	1,	1],						
[	0.13368,	0.13368,	0.17449, ...,	1,	1,	1]]),	'Confidence',	'Precision'],	[array([	0,	0.001001,	0.002002
0.024024,	0.025025,	0.026026,	0.027027,	0.028028,	0.029029,	0.03003,	0.031031,	0.032032,	0.033033,	0.034034,	0.035035,	e
0.048048,	0.049049,	0.05005,	0.051051,	0.052052,	0.053053,	0.054054,	0.055055,	0.056056,	0.057057,	0.058058,	0.059059,	e
0.072072,	0.073073,	0.074074,	0.075075,	0.076076,	0.077077,	0.078078,	0.079079,	0.08008,	0.081081,	0.082082,	0.083083,	e
0.096096,	0.097097,	0.098098,	0.099099,	0.1001,	0.1011,	0.1021,	0.1031,	0.1041,	0.10511,	0.10611,	0.10711,	
0.12012,	0.12112,	0.12212,	0.12312,	0.12412,	0.12513,	0.12613,	0.12713,	0.12813,	0.12913,	0.13013,	0.13113,	
0.14414,	0.14515,	0.14615,	0.14715,	0.14815,	0.14915,	0.15015,	0.15115,	0.15215,	0.15315,	0.15415,	0.15516,	
0.16817,	0.16917,	0.17017,	0.17117,	0.17217,	0.17317,	0.17417,	0.17518,	0.17618,	0.17718,	0.17818,	0.17918,	
0.19219,	0.19319,	0.19419,	0.1952,	0.1962,	0.1972,	0.1982,	0.1992,	0.2002,	0.2012,	0.2022,	0.2032,	
0.21622,	0.21722,	0.21822,	0.21922,	0.22022,	0.22122,	0.22222,	0.22322,	0.22422,	0.22523,	0.22623,	0.22723,	
0.24024,	0.24124,	0.24224,	0.24324,	0.24424,	0.24525,	0.24625,	0.24725,	0.24825,	0.24925,	0.25025,	0.25125,	
0.26426,	0.26527,	0.26627,	0.26727,	0.26827,	0.26927,	0.27027,	0.27127,	0.27227,	0.27327,	0.27427,	0.27528,	
0.28829,	0.28929,	0.29029,	0.29129,	0.29229,	0.29329,	0.29429,	0.2953,	0.2963,	0.2973,	0.2983,	0.2993,	

Importing Test Zip

```
import zipfile

# replace filename if different
zip_path = "Bee _Images.zip"

with zipfile.ZipFile(zip_path, 'r') as zip_ref:
    zip_ref.extractall("/content/")
```

```
import random
import glob

# get all images from your folder
images = glob.glob("/content/Bee _Images/*.jpg")

# pick random test image
test_img = random.choice(images)
```

```
# run detection
results = model(test_img, save=True)

print("Test image used:", test_img)
```

```
image 1/1 /content/Bee _Images/20190506_112058.jpg: 640x480 37 Worker-Bees, 62.9ms
Speed: 5.1ms preprocess, 62.9ms inference, 2.3ms postprocess per image at shape (1, 3, 640, 480)
Results saved to /content/runs/detect/predict
Test image used: /content/Bee _Images/20190506_112058.jpg
```

```
from collections import Counter

labels = results[0].boxes.cls.tolist()
names = model.names

counts = Counter([names[int(i)] for i in labels])

print(counts)
```

```
Counter({'Worker-Bee': 37})
```



Start coding or [generate](#) with AI.

Start coding or [generate](#) with AI.

Requirements

1. Ultralytics
2. Roboflow
3. Pytorch The ultralytics library was used to implement the YOLOv8 object detection model, roboflow was used to manage and import the dataset, and torch (PyTorch) was used as the backend deep learning framework for training and optimisation.

Conclusion

In conclusion, this project successfully developed and evaluated an object detection system using the YOLOv8 model. The results demonstrated strong performance, with the model achieving accuracy in detecting and classifying different bee types and Varroa mites. Through experimentation with various hyperparameters, it was observed that increasing the number of **epochs** improved model performance up to a certain point, after which improvements became minimal, indicating model convergence.

The object counting further enhance the practical value of the system, allowing automatic analysis of detected objects in images. This highlights the real-world applicability of the model in areas such as bee population monitoring and pest detection. Overall, the project demonstrates a solid understanding of deep learning, dataset preparation, model training, and evaluation techniques.